

Pelvyn Robotics
Technical Assessment Submission

Autonomous Underwater Inspection Robot System Architecture & Navigation Framework

Candidate: Kvn Sai Siddartha

Date: June 24, 2026

Platform: ROS 2 Humble | Gazebo 11 | eeUVsim UCAT AUV

1. Executive Summary

This report presents the software and embedded architecture for an autonomous underwater inspection robot (AUV) for submerged infrastructure inspection and mapping. The system integrates forward sonar, IMU, depth control, camera, and odometry within a ROS 2 framework with waypoint mission planning and reactive obstacle avoidance.

A working simulation was built using eeUVsim Gazebo and a custom *pelvyn_inspection* ROS 2 package. The AUV navigates a rectangular path at 2 m depth, detects obstacles via simulated sonar, executes AVOID / AVOID_TURN / AVOID_RECOVER maneuvers, and streams sensor data to RViz and a terminal dashboard.

2. System Architecture

The design separates sensing, embedded control (STM32), onboard computation (Jetson + ROS 2), and ground-station monitoring. Figure 1 shows the full stack.

Figure 1: System Architecture — Autonomous Underwater Inspection Robot

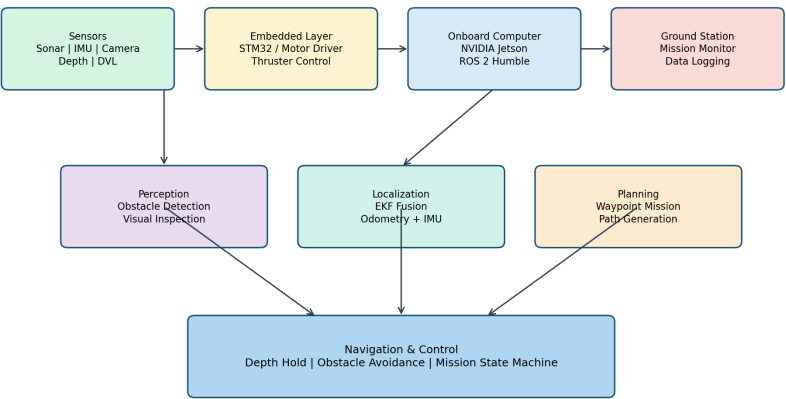


Figure 1: System architecture block diagram.

2.1 Major Modules

Module	Responsibility	Implementation
Perception	Obstacle detection, visual inspection	Sonar raycast + Gazebo camera
Localization	Position and orientation	P3D odometry + IMU + TF
Planning	Mission path	Waypoints in mission_config.py
Navigation	Path tracking, depth hold	stable_mission (10 Hz)
Avoidance	Reactive obstacle response	Sonar threshold state machine
Visualization	Operator display	RViz2 + sensor_monitor

3. Sensor Suite Selection

Sensors support GPS-denied localization, low-visibility obstacle detection, depth hold, orientation tracking, and visual inspection.

Sensor	Purpose	ROS Topic	Simulation / Hardware
Forward Sonar	Obstacle detection	/sonar/forward	Simulated raycast
IMU	Orientation and motion	/ucat/imu_data	Gazebo IMU plugin
Camera	Visual inspection	/camera/image_raw	Gazebo 640×480
Odometry	Horizontal position (DVL equiv.)	/ucat/odom	P3D + kinematic state
Depth	Altitude hold	TARGET_DEPTH	Software lock at -2 m

Production hardware: Teledyne Pathfinder DVL, Bar30 depth sensor, VectorNav VN-100 IMU, multibeam sonar, and low-light HD camera.

4. ROS 2 Software Architecture

ROS 2 Humble runs the *pelvyn_inspection* package for mission control and monitoring. eeUVsim supplies the UCAT URDF and Gazebo plugins (IMU, camera, P3D).

Figure 2: ROS 2 Node & Topic Communication (Simulation)

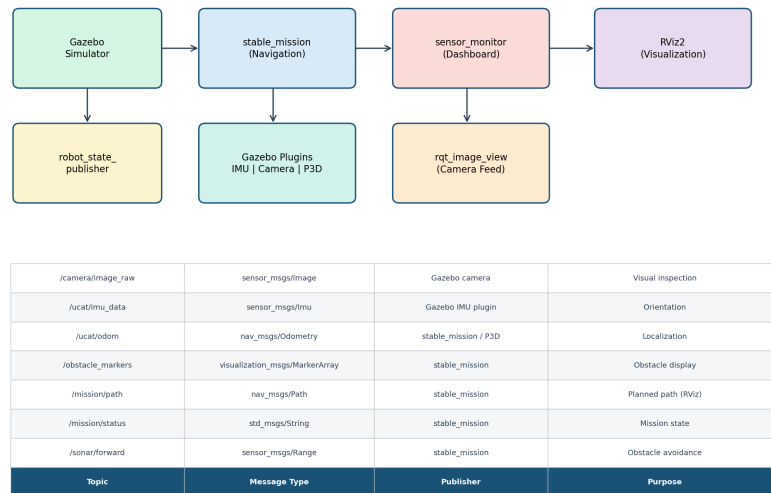


Figure 2: ROS 2 nodes and topics (table below diagram).

4.1 Key Nodes

- **stable_mission** — Waypoint navigation, sonar raycast, avoidance, TF, markers.
- **sensor_monitor** — Terminal dashboard (mission, sonar, IMU, camera) every 1 s.
- **robot_state_publisher** — URDF joint states for RViz.
- **Gazebo plugins** — IMU, camera, P3D odometry.

Launch: *full_demo.launch.py* loads *inspection_world*, spawns UCAT at $(-1, -1, -2)$, starts *stable_mission*, *rqt_image_view*, and RViz2.

5. Navigation Framework

Figure 3: Mission State Machine & Obstacle Avoidance Flow

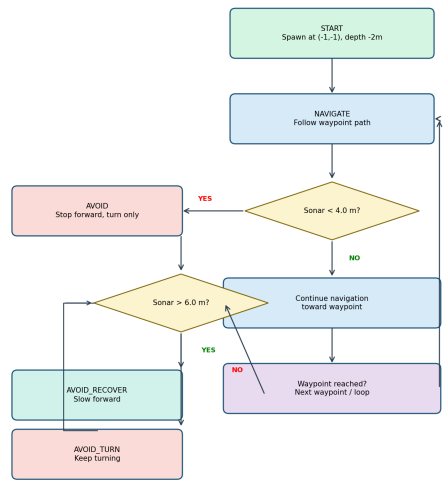


Figure 3: Mission state machine and obstacle avoidance.

Five waypoints at $z = -2.0$ m: $(-1,-1) \rightarrow (5.5,-0.5) \rightarrow (5.5,5.5) \rightarrow (0,5.5) \rightarrow (-1,-1)$. Sonar below 4 m triggers AVOID; above 6 m allows AVOID_RECOVER.

Parameter	Value	Description
SONAR_THRESHOLD	4.0 m	Enter AVOID
SONAR_CLEAR	6.0 m	Safe to recover
ROBOT_RADIUS	0.55 m	Collision envelope
LINEAR_SPEED	0.35 m/s	Cruise speed
YAW_RATE	0.70 rad/s	Turn rate

6. Sensor Fusion Strategy

Figure 4: Sensor Fusion & Perception Pipeline

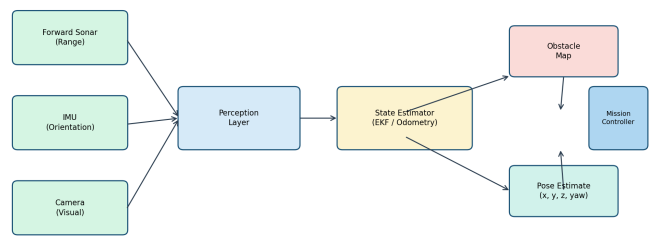


Figure 4: Sensor fusion pipeline.

Simulation uses P3D odometry for pose, IMU for orientation, and sonar for obstacles. On hardware, an EKF would fuse DVL, IMU, and depth at 50–100 Hz.

7. Failure Handling & Safety

- **Sensor loss:** Reduce speed, hold heading, report degraded mode.

- **Depth excursion:** Depth locked at -2.0 m in stable_mission.
- **Collision:** Repulsive force when inside obstacle envelope.
- **Stuck:** AVOID timer and turn alternation; recover when sonar > 6 m.
- **Comms loss:** Watchdog neutralises thrust and triggers ascent after 30 s.

8. Simulation Results

Gazebo 11 tests with four cylindrical obstacles and a dock station. Screenshots show navigation, avoidance, RViz sensor display, and the live terminal dashboard.

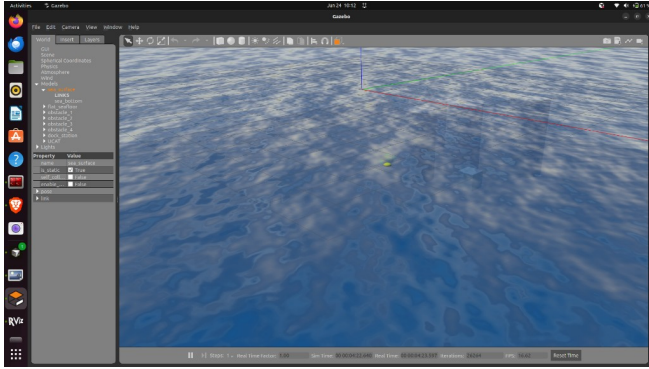


Fig. 5: Gazebo inspection world.

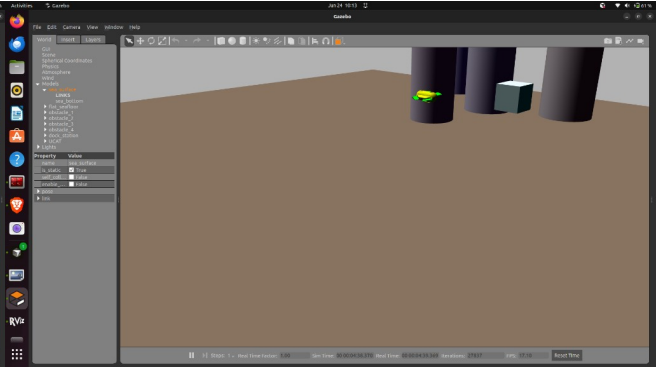


Fig. 6: AUV among obstacles.

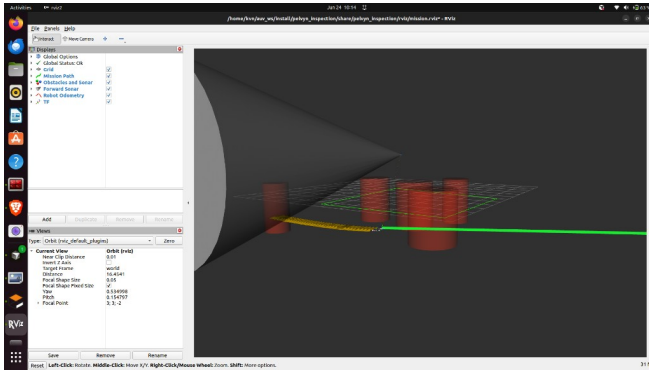


Fig. 7: RViz path, sonar, markers.

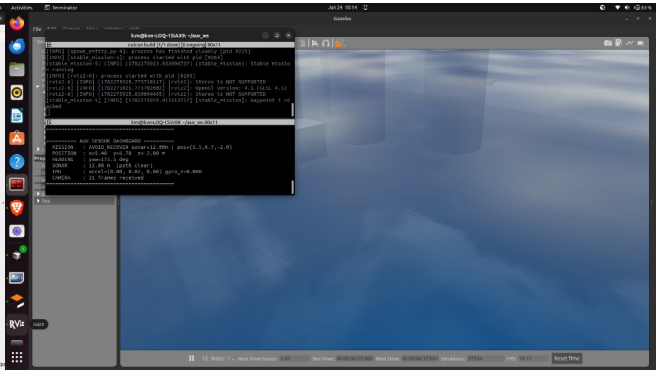


Fig. 8: Terminal dashboard (AVOID_RECOVER).

Observed states: **NAVIGATE**, **AVOID** (sonar < 4 m), **AVOID_TURN**, and **AVOID_RECOVER** (sonar > 6 m). Depth held at $z = -2.00$ m throughout a 6+ minute run.

9. Conclusion

The simulation validates the full inspection stack — sensor integration, waypoint navigation, and sonar-based avoidance — for submerged infrastructure tasks. The AUV completed multiple laps of the inspection path while holding depth and reacting to obstacles in real time.

10. Future Scope

The current work establishes a simulation baseline. The following extensions would move the system toward a deployable inspection platform:

- **Sensor fusion on hardware:** Integrate robot_localization EKF with DVL velocity, IMU angular rates, and Bar30 pressure depth for robust pose estimation at 50–100 Hz.
- **Real sonar interface:** Replace the simulated raycast with a Gazebo ray plugin or a driver for a multibeam sonar (e.g. Oculus M750d) for environment-agnostic obstacle detection.
- **SLAM and mapping:** Add Cartographer or RTAB-Map to build seafloor and structure maps during inspection missions for post-mission analysis.
- **High-fidelity dynamics:** Couple fin/thruster physics with model-predictive control instead of kinematic set_entity_state for realistic hydrodynamic behaviour.
- **Mission operations:** Support mission upload from shore, acoustic or tethered telemetry, and automated data offload after each dive.

- **Docking and recovery:** Extend the dock_station model with autonomous homing and precision docking for battery swap and data retrieval.

Demo command: `ros2 launch pelvyn_inspection full_demo.launch.py` | **Workspace:**
~/auv_ws/src/pelvyn_inspection